

PROJECT DESIGN PHASE

ComicCraft – AI Comic Story Creator using Gemini Models

1. Project Overview

ComicCraft is a web-based application designed to generate personalized comic book stories and illustrations from user-provided prompts. The design combines AI-based storytelling with automated image generation and PDF export, providing an end-to-end comic creation workflow.

2. Design Objectives

The design phase focuses on creating a system that can:

- Accept a story prompt and personalized comic preferences.
- Generate a structured panel-by-panel comic outline.
- Produce narration and character dialogues for each panel.
- Generate comic-style illustrations from image prompts.
- Display the completed comic through a web interface.
- Assemble the generated content into a downloadable PDF.

3. User Inputs

The interface is designed around five primary inputs: Story Prompt, Main Character Name, Setting, Story Tone, and Art Style. These values are passed to the FastAPI backend and used throughout the AI generation workflow.

4. Design Approach

The application follows a modular architecture so that user interaction, backend processing, AI generation, layout construction, and export can be handled as separate but connected stages.

PROJECT DESIGN PHASE – SYSTEM ARCHITECTURE

1. Three Primary Components

The ComicCraft architecture is organized into three primary components: Frontend, Backend, and AI Integration.

Component	Design Responsibility
Frontend HTML, CSS, Jinja2	Collects user inputs and displays the generated comic, panel-by-panel.
Backend FastAPI	Receives inputs, manages routing, coordinates AI calls, builds the layout, and handles PDF export.
AI Integration Gemini + Diffusers	Generates the comic outline, narration, dialogues, and visual illustrations.

2. Frontend Design

The frontend uses HTML, CSS, and Jinja2 templates. The main input page, `index.html`, collects the story prompt, character name, setting, tone, and art style. `comic_preview.html` displays the generated comic, while `export_success.html` confirms the completed export.

3. Backend Design

FastAPI manages server-side operations and routing. The backend receives form data or JSON, calls the AI modules, organizes generated content, renders templates, and exports the final comic.

4. AI Integration Design

Gemini Flash is designed to create a structured 5-panel outline. Gemini Pro expands the outline into detailed narration and character dialogue. Stable Diffusion generates comic-style illustrations from the image prompts.

PROJECT DESIGN PHASE – AI MODEL AND WORKFLOW DESIGN

1. Model Selection

The project document identifies model selection as a key design activity. Models are considered according to creative writing quality, prompt adherence, scene detail, image generation speed and resolution, stability, documentation, and community support.

Model	Purpose in ComicCraft
Gemini Flash models/gemini-1.5-flash	Fast, structured generation of the comic panel outline.
Gemini Pro models/gemini-1.5-pro	Detailed comic narration and engaging character dialogues.
Stable Diffusion runwayml/stable-diffusion-v1-5	Generation of comic-style illustrations from image prompts.

2. Comic Generation Workflow

The designed generation pipeline follows these stages:

1. User enters the story prompt and comic preferences.
2. Gemini Flash generates a structured 5-panel outline.
3. Gemini Pro expands the outline into narration and character dialogues.
4. Stable Diffusion generates an illustration for each panel.
5. The backend matches images with their corresponding panel stories.
6. The layout builder creates the final comic structure.
7. FPDF compiles the panels, images, and narrative into a PDF.

3. Design Principle

The design keeps text generation and image generation as separate modules while connecting them through structured panel information. This modular approach supports a clear and maintainable workflow.

PROJECT DESIGN PHASE – MODULE AND INTERFACE DESIGN

1. Core Module Design

Module / Function	Design Role
gemini_flash.py generate_outline()	Creates the structured 5-panel comic outline.
gemini_pro.py generate_story()	Expands panel outlines into full story narration and dialogue.
image_generator.py generate_image()	Generates and saves panel illustrations using Stable Diffusion.
layout_builder.py build_comic_layout()	Matches panel images with their corresponding story content.
exporters.py save_pdf()	Compiles the completed comic into a multi-page PDF.
routes.py	Connects user requests with the comic generation workflow and templates.

2. API and Route Design

The project defines separate routes for the main user and developer workflows:

- / — loads the homepage.
- /generate — accepts form input and generates the comic.
- /generate-comic/json — accepts JSON and returns layout data and PDF information.
- /export-success — displays the successful export confirmation.
- /test-image — tests image generation using a direct prompt.

3. Template Design

Jinja2 dynamically binds backend data to HTML templates. The comic preview loops through the generated layout and displays panel number, title, image, scene description, captions, and narration.

4. Error Handling Design

The route handlers use FastAPI HTTPException and try-except blocks to handle unexpected failures such as API errors and JSON decoding errors.

PROJECT DESIGN PHASE – DEPLOYMENT AND FINAL DESIGN

1. Development Environment Design

The project is designed to run in an isolated Python virtual environment. Required libraries include FastAPI, Uvicorn, Jinja2, python-multipart, Google Generative AI, Diffusers, Transformers, FPDF, Pillow, and Accelerate.

2. Configuration Design

Sensitive credentials such as the Gemini API key and Hugging Face API token are intended to be stored as environment variables or in a .env file loaded through python-dotenv.

3. Local Deployment Design

The application is launched using Uvicorn with the FastAPI application entry point. The project document specifies local access through `http://127.0.0.1:8000` and API testing through the `/docs` interface.

4. End-to-End Design Summary

ComicCraft is designed as an end-to-end AI-assisted comic creation system. A user begins with a creative prompt and preferences. The backend sends the information through the selected AI generation modules, assembles the resulting content into a panel-based layout, presents the comic through the web interface, and provides a downloadable PDF.

5. Future Design Scope

The project document identifies possible future enhancements such as user profiles, comic libraries, and multi-page story arcs. The current design focuses on single-comic generation while retaining a modular architecture for future expansion.

Conclusion

The Project Design Phase establishes the architecture, model selection, module structure, user interface flow, API routes, AI workflow, and deployment approach required for ComicCraft. The design connects storytelling, illustration, layout, and PDF export into one coherent application workflow.